

ГОВОР ЗА ОДБРАНУ ДИПЛОМСКОГ РАДА

Имплементација обсервабилности микросервисне архитектуре на примеру TravelPlanner апликације

Укупно циљано трајање излагања: ~15 минута (900 секунди) · 16 слајдова

Прочитај овај текст неколико пута унапред — на самој одбрани немој да читаш дословно, него причај својим речима, ослањајући се на кључне тачке.

СЛАЈД 1 — Насловни слајд

Оквирно време: 0:20 минута

Добро дан/вече, поштована комисијо. Ја сам Дајана Булић, а тема мог завршног рада је „Имплементација обсервабилности микросервисне архитектуре, на примеру TravelPlanner апликације”, под менторством др Себастијана Стоје.

Не заборави:

- Изговорити пун наслов рада тачно.
- Кратко, самоуверено — ово је само 20 секунди.

Прелаз: Прелазим одмах на то шта је Travel Planner и зашто сам изабрала баш њега.

СЛАЈД 2 — Travel Planner — постојећа апликација

Оквирно време: 0:55 минута

Travel Planner је постојећа микросервисна веб апликација за планирање путовања. Корисник на једном месту организује цео план путовања — дестинације, дневне активности кроз календар, трошкове и буџет, checklist, подсетнике, а план може да подели са другим особама путем кода или QR кода. Систем је реализован кроз четири одвојена сервиса на Microsoft Service Fabric платформи: Gateway, AuthService, TripService и SharingService.

Не заборави:

- Ово је ПОСТОЈЕЋИ систем — не нешто што сам ја градила од нуле.
- Спомени да ће архитектура детаљно доћи за пар слајдова, сад је само упознавање.

Прелаз: Али чим систем има више сервиса, јавља се један озбиљан проблем — а о томе следећи слајд.

СЛАЈД 3 — Микросервиси су тешки за праћење

Оквирно време: 0:55 минута

Микросервисна архитектура доноси познате предности — независно скалирање, одвојен развој по сервису. Али доноси и озбиљан проблем: један кориснички захтев данас пролази кроз више процеса, база и мрежних позива. Кад нешто крене наопако, питања попут „зашто је овај захтев

спор“, „где тачно долази до грешке“ или „да ли је систем тренутно здрав“ постају знатно теже него код монолитне апликације, где је довољно погледати један логфајл.

Не заборави:

- Нагласи да су ово БАШ она питања на која постојећи Travel Planner НИЈЕ могао да одговори.
- Ово је емотивно/мотивационо језгро рада — не журити овде.

Прелаз: Управо то је разлог зашто сам поставила циљ рада овако.

СЛАЈД 4 — Циљ рада

Оквирно време: 0:40 минута

Циљ рада је био четворострук: прво, анализирати постојећи систем са аспекта обсервабилности и идентификовати конкретне недостатке. Друго, надоградити систем слојем обсервабилности заснованим на индустријском стандарду OpenTelemetry. Треће — и ово ми је било важно — не дирати постојећу пословну логику, планове и трошкове. И четврто, доказати да надоградња стварно ради кроз извршавање, аутоматизоване тестове и мерљиве резултате, а не само кроз инсталацију алата.

Не заборави:

- Нагласи реч „доказати“ — то је нит целог рада и враћа се на крају.

Прелаз: Да бих то урадила, прво сам морала детаљно да анализирам постојеће стање.

СЛАЈД 5 — Архитектура пре надоградње (AS-IS)

Оквирно време: 1:10 минута

Ево архитектуре система пре надоградње. Три сервиса — Gateway, AuthService и TripService — су stateless, што значи да не чувају стање између позива, све се ослања на базу. SharingService је једини stateful сервис, зато што важећи кодови за дељење морају да буду доступни са ниском латенцијом, па се кеширају директно на Service Fabric чвору кроз Reliable Collections. Свака компонента има своју базу, по принципу database-per-service.

Не заборави:

- Не мораш да читаш сваку кутију — покажи руком general flow: Frontend → Gateway → сервиси → базе.
- Кључна реченица: „SharingService је stateful ЗАТО ШТО...” — то показује да разумеш архитектуру, не само да си је нацртала.

Прелаз: Детаљном анализом овог кода дошла сам до пет конкретних недостатака.

СЛАЈД 6 — Идентификовани недостаци

Оквирно време: 1:10 минута

Прво, немогуће праћење захтева кроз више сервиса — не постоји ниједан trace ID. Друго, потпуно одсуство метрика перформанси — CPU, меморија, латенција, стопа грешака. Треће, „тихе“ грешке —

нашла сам конкретан пример у коду где се грешка ухвати, али се нигде не логије, што би оставило систем у недоследном стању без иједног трага зашто. Четврто, ниједан сервис нема health check механизам. И пето, логовање је минимално — грер кроз цео backend показао је нула позива ILogger пре овог рада.

Не заборави:

- Ово су НАЈВАЖНИЈИ слајд за целу причу рада — комисија ће памтити ову листу.
- Ако имаш времена, једном реченицом опиши тихи catch блок (AuthService/TripClient) — то је конкретан доказ да ниси само нагађала.

Прелаз: На основу ове пет тачака, дизајнирала сам решење засновано на једном заједничком стандарду.

СЛАЈД 7 — OpenTelemetry + Grafana stack

Оквирно време: 1:20 минута

Уместо низа појединачних закрпа, изабрала сам OpenTelemetry као централни, vendor-неутралан стандард који обједињује traces, metrics и logs под једним API-јем, са готовом инструментацијом за ASP.NET Core, HttpClient и SqlConnection. Сервиси шаљу телеметрију ка OpenTelemetry Collector-у, који је даље усмерава: Prometheus за метрике, Grafana Tempo за дистрибуирано праћење захтева, Grafana Loki за логове. Grafana је јединствен визуелни слој за све то, са унакрсном навигацијом — из лога директно на trace, из trace-а директно на метрику.

Не заборави:

- Ово је технички најгушћи слајд — успори темпо овде.
- Ако те питају „зашто баш OpenTelemetry а не нпр. Application Insights” — одговор је vendor lock-in, то пише и у раду.

Прелаз: Хајде да видимо како то изгледа кад се дода тај observability слој изнад постојеће архитектуре.

СЛАЈД 8 — ТО-ВЕ архитектура

Оквирно време: 1:10 минута

Ово је циљна архитектура након надоградње. Разлика у односу на претходни дијаграм је управо тај додати observability слој доле лево — Collector, Prometheus, Tempo, Loki и Grafana, који ради кроз Docker Compose, потпуно независно од постојећег Service Fabric кластера. Ниједан сервис није измењен по питању пословне логике — само је додата инструментација.

Не заборави:

- Нагласи: „независно од Service Fabric-а” — то показује да ниси ризиковала постојећи систем.
- „Ниједна линија custom кода за пропагацију контекста” је реченица коју волим да поновим овде ако има времена.

Прелаз: Прва конкретна компонента коју сам имплементирала су health checks — ево како то изгледа.

СЛАЈД 9 — Health checks

Оквирно време: 0:55 минута

Сваки сервис сада излаже два endpoint-a: /health/live, који каже да ли процес уопште живи, и /health/ready, који проверава да ли су критичне зависности спремне — на пример, да ли је SQL база доступна. Gateway нема сопствену базу, па он проверава мрежну доступност три downstream сервиса, преко истих адреса које већ користи YARP за рутирање, без дуплирања конфигурације. Одговор је структурисан JSON, овде видите пример где су сва три сервиса Healthy.

Не заборави:

- Ако питају разлику live/ready — live=процес жив, ready=зависности спремне (Kubernetes-style конвенција).

Прелаз: Health checks су само један део — целу слику добијате тек кад погледате dashboard.

СЛАЈД 10 — Grafana dashboard — све на једном месту

Оквирно време: 0:55 минута

Овај provisioned dashboard приказује RED метрике — request rate, error rate, p95 latency — ресурсе попут CPU-а, меморије и garbage collector-а, и ток логова, све филтрирано по сервису преко template варијабле. Ово директно решава недостатак број два и пет са листе — сада имамо потпун увид у перформансе без иједног ручног прегледа конзоле.

Не заборави:

- Не мораш да објашњаваш сваки график понаособ — довољно је „RED метрике + ресурси, филтрирано по сервису”.

Прелаз: Наравно, dashboard без тестова није доказ — зато прелазим на тестирање.

СЛАЈД 11 — Тестирање — стратегија

Оквирно време: 1:05 минута

Направила сам четири xUnit пројекта, по један за сваки сервис, где сваки тест подиже прав Kestrel host на ефемерном порту, без mock-а базе података. HealthEndpointsTests тестирају health check endpoint-е интеграционо. ObservabilityPipelineTests проверавају да ли се OTel инструментација исправно активира — ту сам, узгред, открила стваран детаљ везан за тренутак када се име руте обогаћује. И TripClientTests су пергесиони тест управо за онај „тихи” catch блок који сам поменула раније. Укупно, свих 21 тест пролази.

Не заборави:

- Ако имаш времена, спомени ObservabilityPipelineTests налаз — то показује дубину анализе, не само писање тестова.
- 21/21 — упамти тај број, враћа се неколико пута.

Прелаз: Али 21 тест који пролази није довољан доказ да тестови нешто ЗАИСТА проверавају — зато сам урадила и ово.

СЛАЈД 12 — Доказ да тестови стварно нешто проверавају

Оквирно време: 0:55 минута

Да бих показала да тестови нису тек формалност, намерно сам привремено вратила оригинални „тихи” catch блок — уклонила сам само један позив LogWarning-а. Резултат: тачно један тест пада, управо онај који проверава да неуспешан HTTP статус производи тај лог, а преостала два теста из истог пројекта остају непромењена и даље пролазе. Након што сам вратила исправку, сви тестови су поново прошли.

Не заборави:

- Ово је „аха” тренутак презентације — говори то помало поносно, ово је твоја идеја да докажеш квалитет тестова.

Прелаз: Осим формалних тестова, урадила сам и четири демонстрациона сценарија над стварним, живим системом.

СЛАЈД 13 — Демонстрациони сценарији над стварним системом

Оквирно време: 1:20 минута

Сва четири сценарија извршена су над стварно постављеним системом, не симулирано. Сценарио 1 — нормалан захтев кроз три сервиса, где trace показује чак девет угнежђених SQL child span-ова. Сценарио 2 — намерно сам угасила TripService, а health check је аутоматски пријавио Unhealthy статус, систем се сам опоравио за седам секунди. Сценарио 3 — k6 load test са 780 захтева, где се клијентска и серверска латенција поклапају готово идентично, 2.12 наспрам 2.14 секунде, што доказује да метрике стварно одражавају понашање система. Сценарио 4 — намерно успорена SQL зависност, где trace тачно локализује да је узрок конкретна UPDATE наредба, не захтев уопштено.

Не заборави:

- Ово је најдужи слајд — ако касниш са временом, скрати сценарио 2 и 3 на по једну реченицу.
- Бројеви 2.12s / 2.14s и 23.9s / 14.5ms су конкретни докази — изговори их прецизно.

Прелаз: Ево како тачно изгледа тај trace из сценарија 1, визуелно.

СЛАЈД 14 — Дистрибуирано праћење захтева у пракси

Оквирно време: 0:55 минута

Овде видите стваран trace из Grafana Tempo-а за сценарио један: захтев иде Gateway, па SharingService, па TripService, и све то се дешава аутоматски, без иједне линије custom кода написане за пропагацију контекста — то долази бесплатно из OpenTelemetry-јеве ASP.NET Core и HttpClient инструментације.

Не заборави:

- Покажи руком водопад-дијаграм лево на десно ако презентујеш уживо.

Прелаз: Ово ме доводи до закључка целог рада.

СЛАЈД 15 — Шта је постигнуто

Оквирно време: 1:10 минута

Да сумирам: идентификовала сам пет конкретних недостатака постојећег система, имплементирала сам надоградњу засновану на OpenTelemetry и Grafana stack-у без измене иједне линије пословне логике, и верификовала сам то на два независна начина — двадесет један аутоматизован тест и четири демонстрациона сценарија над стварно постављеним системом. Што се тиче праваца даљег развоја, ту видим формално alerting решење изнад постојећих dashboard-a, инструментацију frontend-a ради потпуног end-to-end trace-a, и аутоматизацију k6 load теста као дела CI/CD pipeline-a.

Не заборави:

- Ово затвара причу — врати се на циљ рада са слајда 4 ако желиш ефектан завршетак („и управо то сам доказала”).

Прелаз: Хвала на пажњи, ту сам за питања.

СЛАЈД 16 — Хвала на пажњи / Питања

Оквирно време: 0:05 минута

Хвала на пажњи, ту сам за евентуална питања.

Не заборави:

- Стани мирно, насмеши се, сачекај прво питање — не пуни тишину додатним причањем.

Прелаз: (крај излагања)
